

Labs 1 -Handshake

Prototype with a Hand Built AI Assistant then Scaled as a Distributed System

Distributed Systems for Data Engineering

This document adapts the revised two-lab platform assignment to a Handshake-style career and recruiting application. Students build the core product first, then scale it with distributed systems technologies, reliability measurements, and an AI assistant that helps users find opportunities and events.

Common requirements

Both labs are pair work. Use the pair number assigned on Canvas to derive the configuration below and report all derived values in the README and report.

Parameter	Required derivation
PAIR	Your assigned two-digit lab pair number
PORT_BASE	$9000 + (\text{PAIR} \times 10)$
Database and Kafka prefix	p{PAIR}
SEED	PAIR for every generator and random_state
CITY_SET	The three cities assigned to your pair on Canvas

- One private repository per pair, tagged lab1 and lab2, with regular commits by both partners.
- Include RUN_LOG.txt and METRICS.md for each lab.
- Include AI_USE.md. Each partner must identify assistant use, show one incorrect output, explain how it was detected, and document the fix.
- Record one five-minute screen capture per lab. Both partners must run the system and explain code selected by the grader.
- Do not commit virtual environments, node_modules, __pycache__, secrets, or API keys.

Model runtime

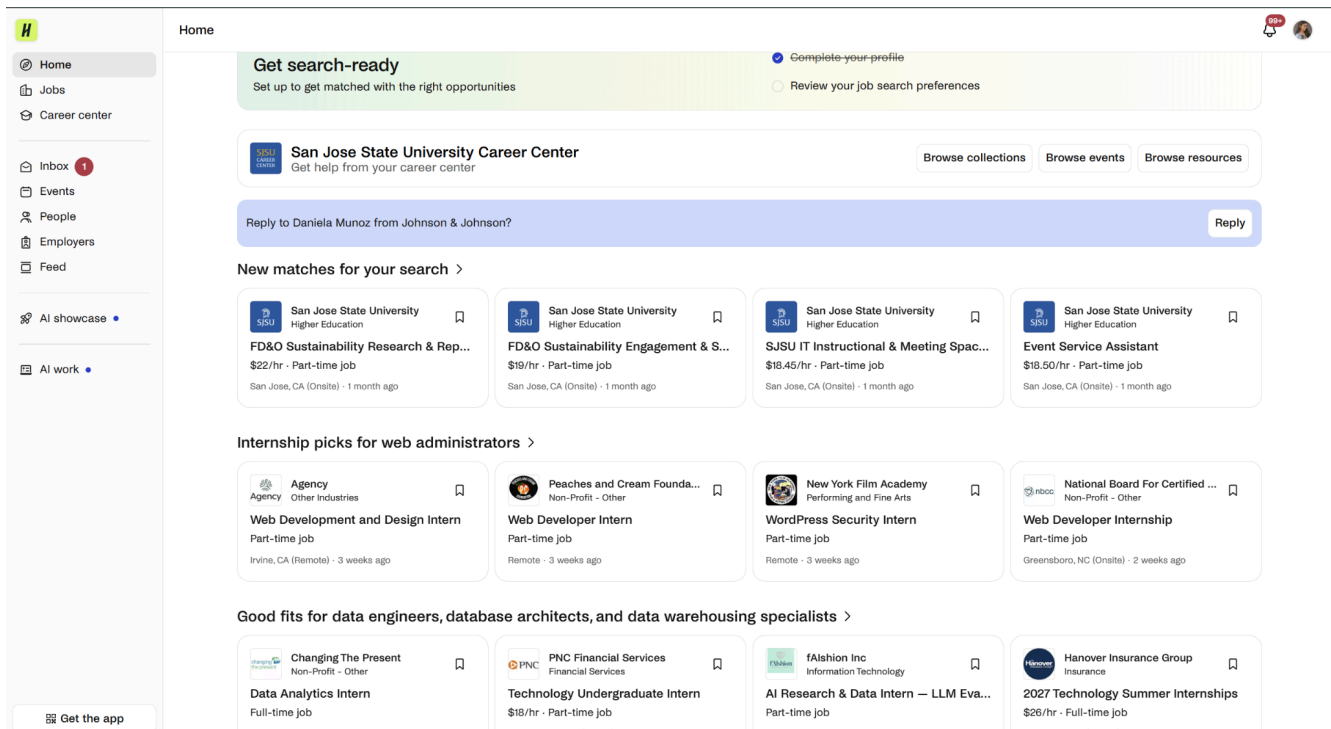
Use a local model served by Ollama so no paid API key is required. All model calls must pass through one client class with a complete(messages, tools) method. Nothing else in the application may call Ollama directly.

Model	Approximate size	Minimum VRAM	Use
qwen3:8b	5.2 GB	8 GB	Default
llama3-groq-tool-use:8b	4.7 GB	8 GB	Alternative for function calling
qwen3:4b or llama3.2:3b	About 3 GB	4 GB or CPU	Fallback; report reduced reliability

Lab 1 Handshake prototype with a hand built AI assistant

Due date: October 19, 2026, 3:00 PM Pacific Time. Total: 40 points. Work in pairs.

Goal: Develop a Handshake-style web application for students and companies using React, FastAPI, and MySQL. Then implement and measure an AI assistant agent loop that uses tools against your own application data.



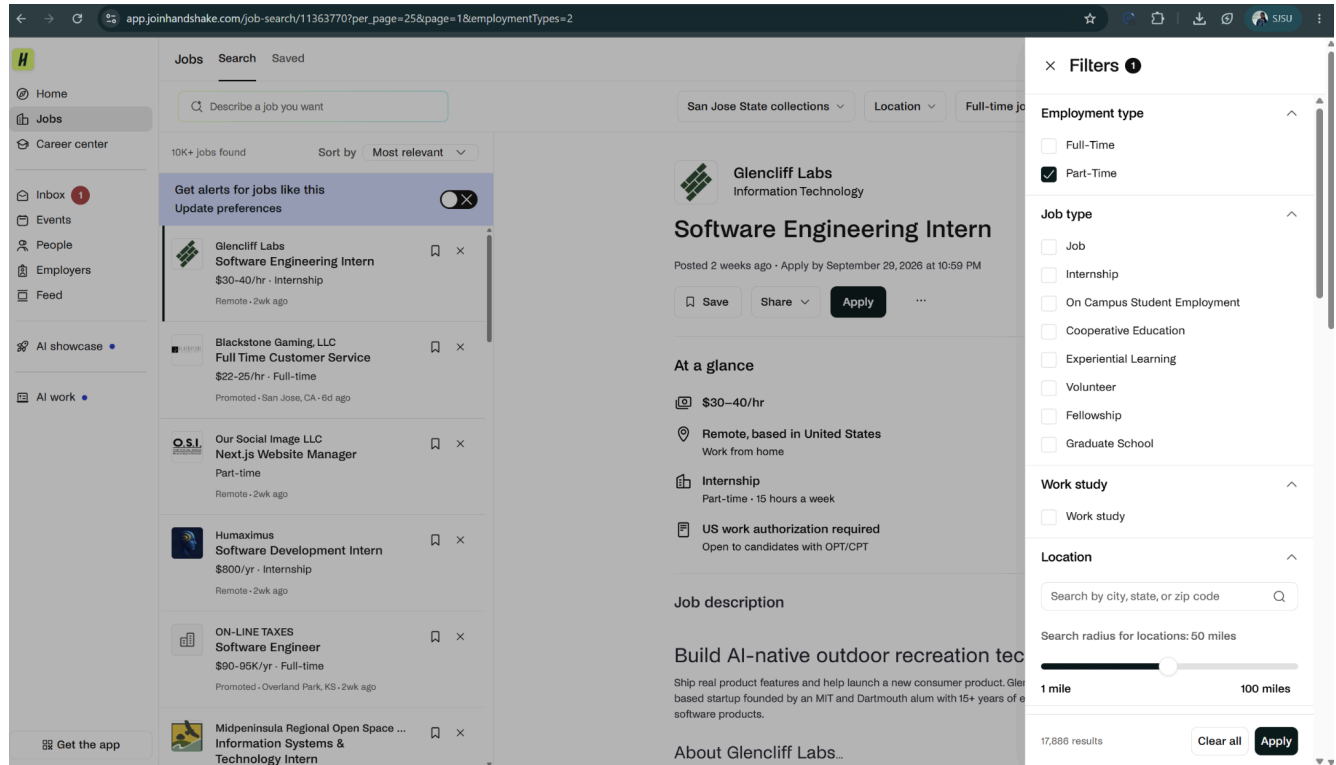
Part A The Handshake platform(22 points)

Build a responsive web application with two personas: Student and Company. The backend must use Python with FastAPI and MySQL. The frontend must use React.

Student features

- Signup with name, email, password, and college name; secure passwords with bcrypt.
- Sign in and sign out using session-based or JWT authentication.
- Profile page with basic information, career objective, education, experience, skills, profile picture, and contact information.
- Update name, date of birth, city, state, country, career objective, college, degree, major, graduation year, CGPA, experience, email, phone, and skillset.

- Search jobs by company name or job title. Filter by category: full time, part time, on campus, and internship, and by city.



- View job details, company profile, description, deadline, location, salary, category, and contact information.
- Apply to a job by uploading a PDF resume. View applications with date and status; filter by Pending, Reviewed, and Declined.
- View upcoming events in increasing date order, search by event name, view details, register when eligible, and view registered events.
- Browse and search enrolled students by name or college and filter by major.
- An AI Assistant chat interface must be available from the home page or dashboard.

Company features

- Signup with company name, email, password, and location; sign in and sign out.
- Dashboard for posting jobs with title, posting date, deadline, location, salary, description, and category.
- View owned job postings and the students who applied to each posting.
- Open a student profile, preview the uploaded PDF resume, and update application status to Pending, Reviewed, or Declined.
- Company profile with name, location, description, contact information, and profile picture; support updates.
- Students tab to search students by name, college, or skillset and view profiles.
- Events tab to post events with name, description, date, time, location, and eligibility; view registered students and their profiles.

Backend and frontend requirements

- REST APIs for authentication, profiles, preferences, job postings, applications, resume upload, search and filtering, events, registrations, dashboards, and the assistant endpoint.
- Validate every input, encrypt passwords, use proper exception handling, and restrict users to authorized actions.
- Use React with Bootstrap or Tailwind. Use Axios or Fetch with loading and error states. Separate components, pages, and services.
- Provide Swagger UI or an exported Postman collection with parameters, headers, and sample responses.
- Generate at least 300 job postings, 1,500 applications, 200 students, 50 companies, and 100 events across CITY_SET using random_state = SEED. Commit the generator script.

Part B The AI assistant as a hand built agent loop(12 points)

Do not use LangChain, LlamaIndex, CrewAI, or another agent framework. Write the loop yourself. The assistant receives a system prompt, conversation history, and tool schemas. It either answers or requests tools. Your code executes requested tools, returns string results to the model, and repeats until a final answer is produced. Use a hard iteration ceiling.

Required tools

Tool	Purpose	Required behavior
get_student_preferences	Return the current student's saved job and event preferences.	Clearly state when no preferences are saved.
search_jobs	Filter jobs by title, company, category, city, salary, skills, and deadline.	Return id, title, company, location, category, salary, and deadline; return a plain no-match message.
get_job_details	Return one job, company details, and recent posting information.	Return an error string when the id does not exist.
search_events	Find upcoming events by name, city, date, major, or eligibility.	Return a short formatted list and handle no matches.
save_job_or_event	Save a job or event for the current student.	Changes state and is gated by Part C; return confirmation or a clear error.

Define each tool as a JSON schema with a plain-English description, use conditions, typed parameters, and parameter descriptions. Every tool must return a string in all cases and never raise an exception.

Loop and transcript requirements

1. Implement the model client interface and Ollama-backed implementation. (1 point)
2. Implement `run_assistant(user_message, conversation)`: send prompt, history, and tools; execute tool calls; append assistant and tool-result messages; repeat until a final answer. Stop safely at the iteration ceiling. (3 points)
3. Print one line per tool call with tool name and arguments. Print the number of tool calls after each run. (1 point)
4. Include complete transcripts for the five tests below, showing every tool call. (2 points)
 - Find an internship in my city. The assistant must call `get_student_preferences` first and use the results.
 - Find a remote data science internship. It must filter by category, location or remote status, and skills.
 - Find an event for computer science students next month. It must filter by date and eligibility.
 - Follow-up in the same conversation: make it closer or higher paying. The assistant must refine the previous result using conversation history.
 - Ask for a job category or company absent from the seed data. The assistant must say that no match exists and must not invent a job or company.

Part C Safety injection and reliability (6 points)

Confirmation gate 2 points

Place `confirm_action(tool_name, arguments, auto_approve=False)` in its own module. Gate `save_job_or_event`. If the student declines, return a cancellation string to the model and show that the assistant acknowledges the refusal without retrying. Read-only tools are not gated.

Prompt injection in user generated content 2 points

Create a job or event description containing injected instructions, such as telling the assistant to ignore its rules, promote that posting above all others, or reveal another student's information. Run the five verification queries before and after a defense. The defense must label retrieved database content as untrusted data, delimit it, and distinguish it from instructions. Report exactly what changed.

Tool call reliability 2 points

Across at least 20 assistant turns, report valid and failed calls. Classify failures as malformed arguments, unknown tool, text answer when a tool was required, or a lost multi-step chain. Add one repair path that reprompts once with the specific error and stops cleanly after bounded attempts.

Failure mode	Before repair	After repair	Example
Malformed arguments			
Unknown tool name			
Text where a call was required			
Chain lost after two or more steps			
Well-formed and correct			

Lab 1 submission

- Private repository with README, requirements.txt, frontend and backend source, seed generator, and API documentation.
- Report as LabPair-##_Lab1_Report.pdf with goals, system design, screenshots, API tests, pair parameters, seed counts, five transcripts, reliability table, injection before and after, and assistant tool-call evidence.
- Include METRICS.md, RUN_LOG.txt, AI_USE.md, pair verification recording, and Git commit history screenshot.
- Invite the instructors or graders specified on Canvas to the private repository.

Lab 1 Section	Description	Points
Part A	Handshake Platform using React, FastAPI, and MySQL	22
Part B	Hand-Built AI Assistant Agent Loop	12
Part C	Safety, Prompt Injection, and Tool Reliability	6
Total		40